

Katholieke
Universiteit
Leuven

**Department of
Computer Science**

RACE CONDITIONS

Session 2

Matt Delalieux (r0890338)
Radi Miah (r2)

Academic year 2016–2017

Contents

1 Implementations of max detection **2**
1.1 Nested for loop (CPU) . . . 2
1.2 Atomic operation (GPU) . . . 2
1.3 Reduction (GPU) . . . 2
1.4 Mutiple blocks . . . 2
1.5 Results . . . 3
1.6 Questions . . . 4
1.6.1 What happens if the input is larger than 2048 (how would you ensure . . . 4
1.6.2 Consider calling the kernel with much less threads than there are data elements. For example, one tenth. How would you ensure all elements get processed? . . . 4
1.6.3 How would you tackle a 2D array? . . . 4

A CUDA Kernels Used in the Report **6**
A.1 Atomic Max Kernel . . . 6
A.2 Single-Block Reduction Kernel . . . 6
A.3 Multi-Block Reduction Kernel . . . 6

1 Implementations of max detection

1.1 Nested for loop (CPU)

The nested for loop was the most straightforward part of the task at hand and was therefore used as a benchmark to check the results were correct for the max detection run on the GPU.

```
for (int i = 0; i < N; i++) {
    if (a[i] > largest) {
        largest = a[i];
    }
}
```

1.2 Atomic operation (GPU)

GPU atomic operations ensure that the read–modify–write sequence is executed atomically, preventing race conditions when multiple threads update the same memory location [1]. In this kernel, each thread processes one element of the input array, similar to how each iteration of the CPU loop processes one element. When multiple threads attempt to update the shared result variable, the hardware serializes these atomic updates to guarantee correctness. The order of these updates is determined by the GPU's scheduling and memory system, not by the thread index. This only works because the operation being applied is max, which is both associative and commutative. This means that the final maximum value is independent of the order in which individual updates are applied.

```
__global__ void atomicMaxing(int *a, int *result, int n) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    if (idx < n){
        atomicMax(result, a[idx]);
    }
}
//Allocate device memory
//Copy data from host to device
atomicMaxing<<<1, N>>>(d_a, largestAtomic, N);
//Copy data from device to host
```

1.3 Reduction (GPU)

Reduction works by recursively halving the number of active elements and threads, with each thread comparing two values and writing the larger one back to the beginning of its segment. After $\log_2(N)$ levels, only a single maximum remains. Each level of the reduction is launched as a separate kernel, and CUDA guarantees that kernels execute in program order: a kernel must finish completely before the next one begins. This implicit global synchronization ensures that all writes from level i are fully completed before any thread from level 2i reads those results. Within a single level, each thread writes to a unique output index, so no two threads ever update the same location. Together, these properties eliminate race conditions entirely.

```
__global__ void compare(int *a, int n, int lvl) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    if (lvl*idx + lvl/2 < n){
        if (a[lvl*idx] < a[lvl*idx+lvl/2]) {
            a[lvl*idx] = a[lvl*idx+lvl/2];
        }
    }
}
for(int i = 2; i<=N*2; i=i*2){
        compare<<<1, 1+N/i>>>(d_a, N, i);
    }
```

1.4 Mutiple blocks

Since each block is limited to 2048 threads, a naive approach of assigning one thread per element would break down for inputs larger than 2048 elements per block. Our implementation handles this through two phases.

In Phase 1, each thread iterates over multiple elements using a strided loop — thread 0 processes elements 0, 1024, 2048, ..., thread 1 processes elements 1, 1025, 2049, and so on for a block with 1024 threads. Each thread computes a local maximum across all its assigned elements and stores that single value into shared memory. After this phase, no matter how large the input is, we have reduced it down to at most 1024 values in shared memory — one per thread.
In Phase 2, we perform a standard pairwise reduction over those shared memory values to find the block's maximum. This phase only ever operates on at most 1024 values, so it fits entirely within one block's shared memory and only requires __syncthreads() calls to stay correct. Since we launch multiple blocks, each block independently produces one maximum value and writes it to a block_results array in global memory. Once all blocks have finished, the final answer is found by taking the maximum over block_results, which in our implementation is done on the CPU. This inter-block coordination is handled naturally by cudaDeviceSynchronize() ensuring all blocks complete before the CPU reads the results — no additional in-kernel synchronization across blocks is needed. This design means the implementation scales correctly to any input size without hitting CUDA's thread-per-block limit.

1.5 Results

Figure 1 shows clearly that the GPU has more overhead, causing it to perform worse for small input sizes. After around 5000 indexes the GPU begins to outperform the CPU, with AtomicMax performing slightly worse than the reduction approach for even larger arrays. Each measurement is the average of 100 runs.

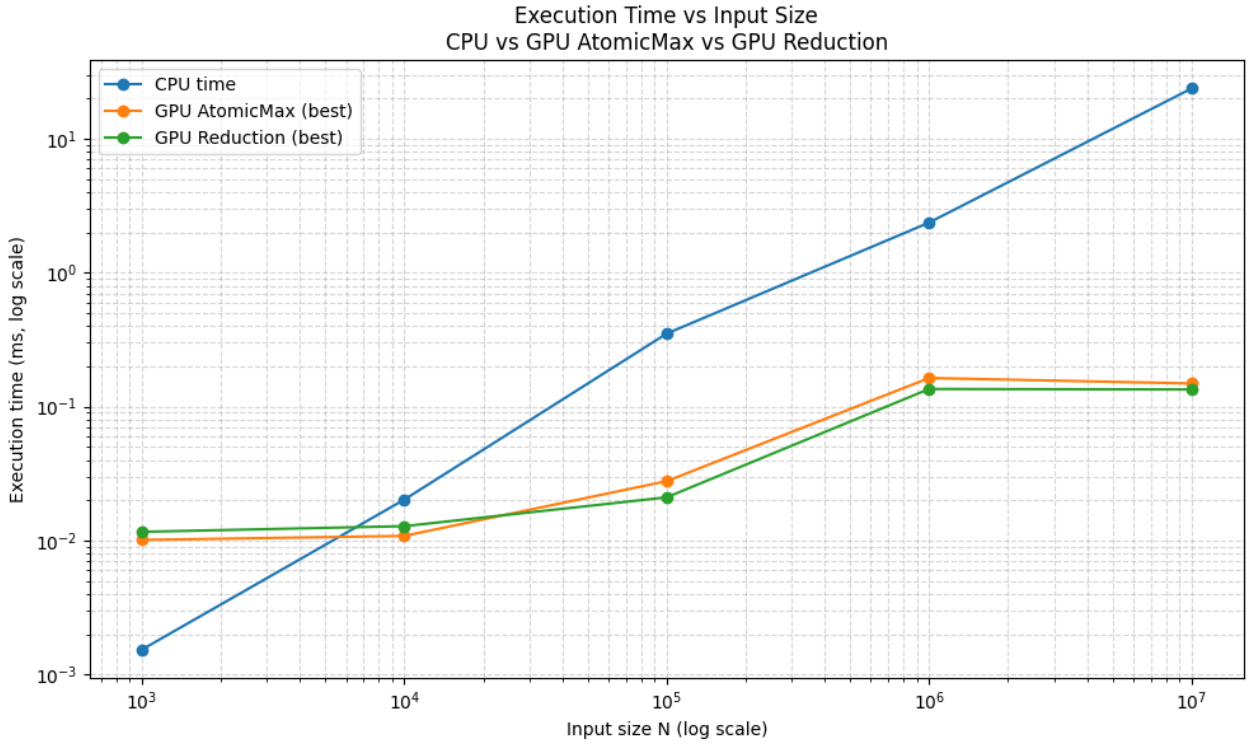


Figure 1: Performance comparison of CPU, GPU AtomicMax, and GPU Reduction

Figure 2 show that the number of blocks is low, configurations with more threads per block perform better because each block carries more work and the GPU stays busier. As the block count grows, the GPU quickly reaches its limit on how many blocks can run at once. Beyond that point, adding more blocks doesn't increase parallelism anymore, and smaller thread-block sizes start to benefit slightly from lower per-block overhead, which is why the curves begin to converge and even reverse at high block counts. The same idea can be seen in figure 3.

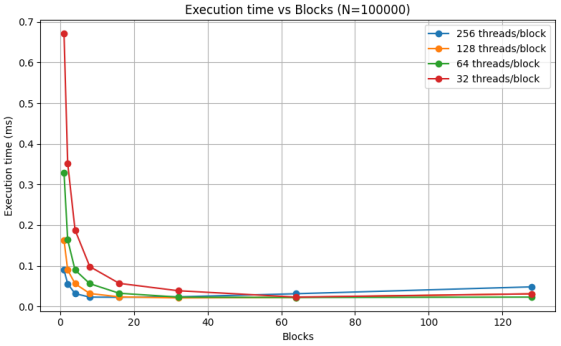


Figure 2: Execution vs Blocks

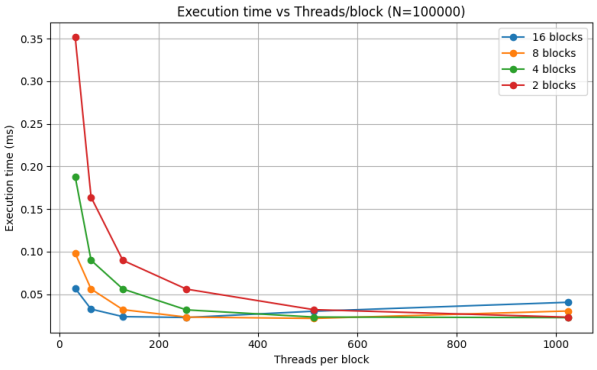


Figure 3: Execution vs Threads per Block

1.6 Questions

1.6.1 What happens if the input is larger than 2048 (how would you ensure

correct calculation and are there additional synchronization steps)?

The CUDA thread limit is 1024 per block, considering that the reduction implementation uses 1 thread per 2 input values, an input larger than 2048 would require more blocks. CUDA provides no mechanism for synchronizing threads across blocks within a kernel[1]. Without additional synchronization or a multi-kernel hierarchical reduction, threads may read stale values, leading to incorrect results.

1.6.2 Consider calling the kernel with much less threads than there are data elements. For example, one tenth. How would you ensure all elements get processed?

Because each element is considered by the thread with the same idx, there must be at least half the amount of threads as elements in the array for reduction. Having only one tenth of the threads as elements we would see the maximum of the first twenty percent of the array. If one was to again implement the idea of stride, jump forward the amount of threads that are being used each time, stopping once the end of the array is reached, this issue could be solved.

1.6.3 How would you tackle a 2D array?

To process or reduce a 2D array efficiently, the strategy depends heavily on how the data is laid out in memory. We propose three primary approaches:

- **Flattening to 1D:** The array is treated as a single contiguous block of memory. This allows for the use of a standard 1D kernel, which is simple to implement but may lose dimensional context if row-specific or column-specific operations are required.
- **One Block per Row:** A single thread block is assigned to handle each row. This is highly effective if the data is stored in **Row-major order** (contiguous rows), as it ensures threads reads memory next to each other and ensures great cache locality. After each block performs a reduction on its row, the remaining results can be processed via a CPU loop or a second GPU kernel.
- **One Block per Column:** A single thread block is assigned to each column. This is the preferred method if the data is stored in **Column-major order**.

Memory Considerations: The choice between assigning blocks to rows versus columns is dictated by **contiguous memory alignment**. If rows are contiguous (Row-major), Option 2 is superior because threads within a warp will access adjacent memory addresses, maximizing throughput. Conversely, if columns are contiguous, Option 3 should be used to avoid strided memory access, which significantly degrades performance.

[1]Modern CUDA (since 6.0) offers Cooperative Groups (grid.sync()), which does allow grid-wide synchronization, but only under specific constraints (the grid must fit entirely in the GPU, launched via cudaLaunchCooperativeKernel). So the claim that "CUDA provides no mechanism" is slightly outdated — it's more accurate to say there's no general-purpose, unconditional cross-block sync.

References

[1] NVIDIA Developer Forums. *Where do atomic operations go and why are atomics to shared faster than those to gmem?* Accessed: 2026-02-24. 2022. URL: https://forums.developer.nvidia.com/t/where-do-atomic-operations-go-and-why-are-atomics-to-shared-faster-than-those-to-gmem/220300.

A CUDA Kernels Used in the Report

A.1 Atomic Max Kernel

Listing 1: Atomic max kernel

```
__global__ void atomicMaxing(int *a, int *result, int n) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    if (idx < n) {
        atomicMax(result, a[idx]);
    }
}
```

Launch example:

```
atomicMaxing<<<1, N>>>(d_a, d_result, N);
```

A.2 Single-Block Reduction Kernel

Listing 2: Single-block level-by-level reduction

```
__global__ void compare(int *a, int n, int lvl) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;

    if (lvl * idx + lvl / 2 < n) {
        int left  = lvl * idx;
        int right = left + lvl / 2;

        if (a[left] < a[right]) {
            a[left] = a[right];
        }
    }
}
```

Launch example:

```
for (int lvl = 2; lvl <= N; lvl *= 2) {
    int threads = (N + lvl - 1) / lvl;   // one thread per pair
    compare<<<1, threads>>>(d_a, N, lvl);
}
```

A.3 Multi-Block Reduction Kernel

Listing 3: Multi-block reduction using shared memory

```
__global__ void reducePerBlock(int *input, int *block_results,
                               int n, int itemsPerBlock)
{
    extern __shared__ int sdata[];
    int tid = threadIdx.x;
    int blockStart = blockIdx.x * itemsPerBlock;

    // Phase 1: local max per thread
    int localMax = INT_MIN;
    for (int i = tid; i < itemsPerBlock && (blockStart + i) < n; i += blockDim.x) {
        int val = input[blockStart + i];
        if (val > localMax) localMax = val;
    }

    sdata[tid] = localMax;
    __syncthreads();

```

```
    // Phase 2: shared-memory reduction
    for (int stride = 1; stride < blockDim.x; stride *= 2) {
        if (tid % (2 * stride) == 0 && tid + stride < blockDim.x) {
            if (sdata[tid] < sdata[tid + stride]) {
                sdata[tid] = sdata[tid + stride];
            }
        }
        __syncthreads();
    }

    if (tid == 0) {
        block_results[blockIdx.x] = sdata[0];
    }
}
```

Launch example:

```
int threads = 256;
int blocks  = (N + itemsPerBlock - 1) / itemsPerBlock;
int sharedMem = threads * sizeof(int);

reducePerBlock<<<blocks, threads, sharedMem>>>(
    d_input, d_blockResults, N, itemsPerBlock
);
```